SolutionArchitectAI. Ключевые технические решения

Технические решения

Содержание

1. Система и границы 1
2. Backend и модель 1
3. Хранение, редакции и восстановление 2
4. LLM, агенты и граф 2
5. Frontend, качество и поставка 3

Статус: рекомендуемая архитектура MVP для предметного ревью, а не отчёт об испытании приложения. Реализация backend/frontend и испытания конкретных LLM ещё впереди. Развёрнутые основания и контракты — в технической документации.

1. Система и границы

Уровень	Ответственность
React / FastAPI	Рабочее место архитектора, чаты, API, запуск и наблюдение за операциями
Application	Use cases, права, привязка задач, запусков, Git-редакций и согласований; предметные команды
LangGraph и агенты	Выполнение многоэтапного процесса; роли извлекают требования, задают вопросы, проектируют и независимо проверяют
LangChain / Domain	Обращение к LLM и структурная валидация / архитектурные объекты, операции и инварианты
Persistence / Infrastructure	PostgreSQL, Git, файлы, LLM-адаптеры и рендеринг

Стрелки работы: UI → FastAPI → Application → LangGraph → агенты → LangChain → локальная LLM. Application обращается к Domain и портам хранения. Граф ведёт исполнение процесса, но не становится источником истины архитектурной модели. Предложения ИИ проходят предметную валидацию и решение человека. Применение изменения, согласование результата и Git merge остаются разными операциями с проверкой полномочий.

2. Backend и модель

Рекомендованный стек: FastAPI/OpenAPI для HTTP и streaming, Dishka для scoped зависимостей, Pydantic для внешних DTO и Settings, SQLAlchemy 2.x async + Alembic для PostgreSQL. Выбор ещё не подтверждён эксплуатацией этого приложения; совместимые версии закрепляются в lock-файлах при реализации. Асинхронные клиенты, сессии и

checkpointer управляются через `async with` и lifespan; конкурентные операции не делят одну `AsyncSession`.

`domain` содержит Python-модель архитектурных объектов, отношений, решений и разрешённых операций с инвариантами. JSON служит сериализации, а не источником бизнес-правил. `application` содержит use cases, агентные контракты и порты к внешним средствам; `persistence` реализует SQL/Git/files, `infrastructure` — LLM и рендеринг, `presentation` — API, `config` — загрузку настроек, `di` — композицию. Зависимости направлены к Application/Domain. Protocol/ABC создаются при реальной границе или замене, а не для каждого класса.

3. Хранение, редакции и восстановление

Средство	Авторитетные данные
PostgreSQL	Задачи, чаты, сообщения, запуски, права, ответы, предложения и журнал предметных операций; отдельно checkpoints LangGraph
Git	Подтверждённые редакции общей архитектурной модели, дерево представлений, решения и проверяемая история изменений
Local files / S3-compatible	Неизменяемые версии входных документов, вложений и крупных производных файлов

Задача создаёт рабочую Git-ветку от точной редакции общей модели. Применение предложения требует `operation_id`, ожидаемого HEAD, повторной проверки входов и разрешённых Domain-операций. PostgreSQL-журнал фиксирует подготовку, публикацию и итог; при потере ответа исход восстанавливается по журналу и Git ref без второго commit. Checkpoint графа не заменяет этот журнал. Общей транзакции PostgreSQL/Git/S3 нет: частичный сбой требует идемпотентного восстановления. Approval и merge выполняет уполномоченный человек отдельно.

4. LLM, агенты и граф

Для базовых интерфейсов сообщений/цепочек выбран `langchain-core`, для настраиваемого OpenAI-совместимого endpoint — `langchain-openai` (`ChatOpenAI`). Полный пакет `langchain` базовому MVP не нужен. LM Studio рассматривается для разработки, vLLM — как следующий сервер; одинаковый HTTP-формат не гарантирует поддержки JSON Schema и прочих возможностей конкретной модели. Эти пары модель/сервер подлежат интеграционному испытанию.

StructuredOutputGenerator — собственный переиспользуемый компонент за портом Application. Он принимает промпт/сообщения, профиль модели и Pydantic-схему, возвращает типизированный результат либо типизированную ошибку. `with_structured_output(..., method="json_schema")` допускается только при проверенной поддержке. Иначе явно настроенный режим JSON-текста всё равно проходит строгую Pydantic-валидацию. Невалидный JSON и ошибка схемы допускают контролируемую повторную генерацию. `RetryConfig` задаёт один бюджет запросов и времени для сетевых, rate-limit и validation повторов; вложенные независимые retry-loop запрещены. Отмена и отсутствие capability завершаются явной ошибкой. Автоматического перехода к Tool Calling и молчаливого ослабления схемы нет. LangChain не воспроизводит сам все механизмы исправления ответа Instructor.

Для многоэтапного проектирования выбран `langgraph` с детерминированным `StateGraph`. Он управляет зависимостями, параллельными ветвями, checkpoint, ожиданием человека и возобновлением. Это заменяет прежний план собственного движка переходов. LiteLLM и Instructor не входят в базовый стек, но могут быть пересмотрены по результатам экспериментов. Решение учитывает существующее применение LangChain/LangGraph в организации; работоспособность в данном приложении пока не испытана.

Сохраняются четыре роли: RequirementsExtractionAgent, ClarificationAgent, ArchitectureDesignAgent, ArchitectureReviewAgent. Их контракт — `AgentRequest → AgentResult`;

узел графа адаптирует результат к GraphState. Проектирование разбито на контекст, reuse, компоненты, данные, потоки, взаимодействия, сборку кандидата, представления и независимое ревью. Один агент может выполнить несколько операций; отдельный агент для каждого узла не требуется.

`langgraph-checkpoint-postgres` сохраняет компактный GraphState в PostgreSQL: run ID, версии входов, Git base revision, ссылки на результаты и данные для продолжения. Полная архитектура остаётся в своих хранилищах. Для async backend рассматривается `AsyncPostgresSaver` с управляемым lifespan, явным `setup()` до первого использования и строгой политикой десериализации. После `interrupt()` узел при resume может начаться заново: внешние действия защищаются идемпотентностью. LangGraph не определяет семантическую неактуальность предложений после смены требований; Application/Domain проверяют версии и выбирают затронутые операции для повтора.

5. Frontend, качество и поставка

React + TypeScript + Vite, React Router и TanStack Query образуют рекомендуемую основу SPA. Для чата рекомендуется assistant-ui с собственным транспортом FastAPI; история и статусы хранятся на сервере. Интерактивные архитектурные представления показывают общие объекты, их связи, свойства, использование в разных видах и diff предложений. Layout вида отделён от семантики модели. Конкретная библиотека графа и корпоративная UI-библиотека выбираются после короткого испытания; Light/Dark/System входят в MVP.

Ruff, Pyright, pytest и import-linter проверяют Python-код и границы импортов; ESLint, Vitest и Playwright — frontend и сквозные сценарии. Docker Compose поставляет frontend, backend, PostgreSQL и постоянные тома; inference server развёртывается отдельно. Первый рабочий результат — сохраняемый чат React → FastAPI → LangChain → локальная LLM без LangGraph. Следующий — извлечение требований, вопрос архитектору, прерывание и возобновление через LangGraph. Позднее добавляются проектирование и review. Реальные проверки модели, повторов, восстановления после рестарта, конкурентности и Git-идемпотентности относятся к последующим PR, а не к этому документу.